UNIT - 3

TOPICS

Data Constraints and Functions

- Pseudo columns – ROWID, ROWNUM, USER, UID, SYSDATE
- Null values, TAB table, DUAL table
- Operators – arithmetic, relational, logical, range searching, pattern matching and set
- Data constraints – Introduction, advantages and disadvantages
- Type of data constraints – NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY and CHECK
- Modifying constraints, working with data dictionary and use of USER_CONSTRAINTS
- Functions – introduction, merits and demerits, types of functions (scalar and aggregate)
- Scalar : Numeric functions (ABS, FLOOR, MOD, POWER, ROUND, SIGN, SQRT and TRUNC), Character functions (CHR, ASCII, CONCAT, INITCAP, LOWER, SUBSTR,TRIM, UPPER), Date functions (ADD_MONTHS, LAST_DAY, NEXT_DAY, MONTHS_BETWEEN), Conversion functions (TO_NUMBER, TO_CHAR and TO_DATE)
- Aggregate fun : AVG, COUNT, MAX, MIN, SUM
- Miscellaneous functions – NVL, DECODE, COALESCE

PSEUDO COLUMNS: A **pseudo-column** is an Oracle assigned value (pseudo-field) used in the same context as an Oracle Database column, but not stored on disk.

ROWID : ROWID returns the rowid (binary address) of a row in a database table which will be the unique. You can use variables of type UROWID to store rowids in a readable format. f.e. (Select rowid, eno from emp;)

ROWNUM: ROWNUM returns a number indicating the order in which a row was selected from a table. The first row selected has a ROWNUM of 1, the second row has a ROWNUM of 2, and so on. If a SELECT statement includes an ORDER BY clause, ROWNUMs are assigned to the retrieved rows before the sort is done.

You can use ROWNUM in an UPDATE statement to assign unique values to each row in a table. Also, you can use ROWNUM in the WHERE clause of a SELECT statement to limit the number of rows retrieved, as follows:

```
SELECT empno, sal FROM emp
    WHERE sal > 2000 AND ROWNUM < 10; -- returns 9 rows
```

USER : Returns the user name of a database user f.e (Select user from Dual)

UID : Return the User ID of a database user. F.e. (Select uid from Dual)

SYSDATE : Returns the current system date. f.e. (Select Sysdate from Dual)

DUAL Table :-

This table is owned by the SYS user and can be accessed by all users. It contains one column, Dummy, and one row with the value 'X'. The DUAL table is useful when you want to return a value once only, for instance, the value of a constant, Pseudocolumn, or expression that is not derived from a table with used data. The DUAL table is generally used with SELECT clause for syntax completeness.

TAB TABLE:

This table contains a list of all of the tables in a database. To list table in a database you can write a query like Select * from Tab;

NULL VALUE: A column in a row contains Null value in oracle is treated as special one which indicates the column in that row contains no value.

•Null is not same as zero. As zero has a value but null represent no value.

•Setting a null value is appropriate when the actual value is not known or when a value would not be meaningful.

•Currently oracle Database treats a character value with a length of zero as null which is exampled in section below. But according to oracle this behavior may change in future.

•If we do any arithmetic operation of a NULL value then it evaluates to null.

•All operators (except concatenation) return null when given a null operand.

•All scalar functions (except REPLACE, NVL, and CONCAT) return null when given a null argument.

•Most aggregate functions ignore nulls. For example in a column I have 11,null,12 values. Now if I do an average of the column it will ignore the null value and will result average with other column as (11+12)/2.

OPERATORS

ARITHMETIC OPERATOR :

Oracle allows arithmetic operators to be used while viewing records from a table or while performing data manipulation operations such as INSERT, UPDATE, DELETE.

+ :- Addition - :- Subtraction / :- Division

* :- Multiplication ( ) :- Enclosed operation has first priority.

Example:-

Select Sal + Comm 'Total Salary' , sal * 12 'Annual Salary', Sal/25 'Per day salary' , sal - 500 from emp;

RELATIONAL OPERATOR :

=	:- Equal to	>	:- Grater Than
>=	:- Greater than or Equal to	<	:- Less Than
<=	:- Less Than or equal to	< >, !=, ^=	:- Not equal to

These all are known as comparison conditions also These ar used in conditions that compare one expression to another value or expression. They are used in where clause and with conditional statement. An alias name can not be used in WHERE clause.

Example : Select * from emp where sal >= 1500;
Select * from emp where sal <> 1500;

LOGICAL OPERATOR

A logical condition combines the result of two component conditions to produce a single result based on them or inverts the result of a single column. A row is returned only if the overall result of the condition is true.

You can use several conditions in one WHERE clause using the AND and OR operator.

→ **AND** :- The AND operator allows creating an SQL statement based on two oor more conditions being met. It can be used in any valid SQL statement such as SELECT, INSERT, UPDATE or DELETE. The AND operator requires that Both the conditions to be true.

→**OR** :- The OR operator allows creating an SQL statement where records are returned when any one of the conditions are met means either conditions to be true included in the result set. It can be used in any valid SQL statement such as SELECT, INSERT, UPDATE or DELETE.

Example :- Select * from emp where sal >= 1500 AND sal <= 2000
Select * from emp where design= 'Manager' OR design='Clerk';

→ **NOT operator** :- The oracle engine will process all rows in a table and display only those records that do not satisfy the condition specified.

RANGE SEARCHING :

BETWEEN : You can display rows based on a range of values using the BETWEEN range condition. Values specified with BETWEEN condition are inclusive. You must specify the lower limit first. Two values in between the range must be linked with keyword AND. The BETWEEN operator can be used with both character and numeric datatypes.

Syntax:- BETWEEN <lower value> AND <higher value>

Example :- Select * from emp where sal BETWEEN 1200 AND 2000 ;

IN :- To test for values in a specified set of values, use the IN condition is also known as the membership condition. The IN condition can be used with any data type. If Character or Date are used in the list, They must be enclosed in single quotation marks. (' ')

Example :- Select * from emp where deptno in (10,20,30)
Select * from emp where design in ('Manager','Clerk')
Select * from emp where hiredate in ('10-Jan-08', '28-Dec-06')

PATTERN MATCHING :

You may not always know the exact value to search for. You can select rows that match a character pattern by using the LIKE condition The character pattern matching operation is referred to as a wild card search.

(%) → Represents any sequence of zero or more character.
(_) → Represents any single character.

The LIKE condition can be used as a shortcut for some between comparison. The % and _ symbols can be used in any combination with literal character.

Example :- Select * from emp where ename like _a%;

Above example display the employee information whose name have the second character as 'a'.

SET OPERATORS: THE UNION, INTERSECT AND MINUS (By using following tables)

EMPLOYEES :-

Name	Null?	Type
EMPLOYEE_ID	NOT NULL	NUMBER (6)
FIRST_NAME		VARCHAR2 (20)
LAST_NAME	NOT NULL	VARCHAR2 (25)
EMAIL	NOT NULL	VARCHAR2 (25)
PHONE_NUMBER		VARCHAR2 (20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2 (10)
SALARY		NUMBER (8,2)
COMMISSION_PCT		NUMBER (2,2)
MANAGER_ID		NUMBER (6)
DEPARTMENT_ID		NUMBER (4)
DEPARTMENT_NAME		VARCHAR2 (14)

JOB_HISTORY :-

Name	Null?	Type
EMPLOYEE_ID	NOT NULL	NUMBER (6)
START_DATE		DATE
END_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2 (10)
DEPARTMENT_ID		NUMBER (4)

- **UNION :-** The UNION operator returns all rows selected by either query. Use the UNION operator to return all rows from multiple tables and eliminate any duplicate rows.
  - Guideline :-
    1) The number of column and the data types of the columns being selected must be identical in all the SELECT statements used in the query. The names of the columns need not be identical.
    2) UNION operates over all of the column being selected.
    3) NULL values are not ignored during duplicate checking.
    4) The IN operator has a higher precedence than the UNION operator.
    5) By default, the output is sorted in ascending order of the first column of the SELECT clause.

```
SELECT employee_id, job_id FROM employees
UNION
SELECT employee_id, job_id FROM job_history;
```

The UNION operator eliminates any duplicate records. If there are records that occur both in the EMPLOYEES and the JOB_HISTORY tables and are identical, the records will be displayed only once.

```
SELECT employee_id, job_id, department_id FROM employees
UNION ALL
SELECT employee_id, job_id, department_id FROM job_history ORDER BY employee_id;
```

- **INTERSECT :-** It used to return all rows common to multiple queries.
  - Guidelines :-
    1) The number of columns and the data types of the columns being selected by the SELECT statement in the queries must be identical in all the SELECT statements used in the query. The names of the columns need not be identical.
    2) Reversing the order of the intersected tables does not alter the result.
    3) INTERSECT does not ignore NULL values.

```
SELECT employee_id, job_id, department_id FROM employees
INTERSECT
SELECT employee_id, job_id, department_id FROM job_history;
```

The query returns only the records that have the same values in the selected columns in both tables.

- **MINUS :-** Use the MINUS operator to return rows returned by the first query that are not present in the second query (the first SELECT statement MINUS the second SELECT statement.)
  - Guideline :-
    1) The number of columns and the data types of the columns being selected by the SELECT statements in the queries must be identical in all the SELECT statements used in the query. The names of the columns need not be identical.
    2) All of the columns in the WHERE clause must be in the SELECT clause for the MINUS operator to work.

```
SELECT employee_id, job_id FROM employees
MINUS
SELECT employee_id, job_id FROM job_history;
```

CONSTRAINTS : Constraints are used to limit the type of data that can go into a table. Constraints can be specified when a table is created (with the CREATE TABLE statement) or after the table is created (with the ALTER TABLE statement).
The oracle server uses constraints to prevent invalid data entry into tables.

USES OF CONSTRAINT :- You can use constraints to do the following

1) Enforce rules on the data in a table whenever a row is inserted, updated or deleted from that table. The constraint must be satisfied for the operation to succeed.
2) Prevent the deletion of table if there are dependencies from other tables.
3) Provide rules for oracle tools such as oracle developer.

All constraints are stored in the data dictionary. Constraints are easy to reference if you give them a meaningful name. If you do not name your constraint the oracle server generates a name with format SYS_Cnnnnn . where n is an integer so that the constraint name is unique.

→ Constraints can be defined at the time of table creation or after the table has been created.
→ You can view the constraints defined for a specific table by looking at the USER_CONSTRAINTS data dictionary table.
f.e. **Select constraint_name, table_name, search_condition from user_constraints**;

Constraints can be defined at one of two levels.
→ **COLUMN LEVEL :-** References a single column and is defined within a specification for the owning column can define any type of integrity constraints.

→ **TABLE LEVEL :-** References one or more columns and is defined separately from the definitions of the column in the table ; can define any constraints except NOT NULL.

Syntax:

Create table <table name>
(Column name <data type and size> default <expr> <column constraints>,
……. ,
…….,
<Table level constraints>
)

VIEWING CONSTRAINTS :-

After creating a table you can confirm its existence by issuing a DESCRIBE command. The only constraint that you can verify is the NOTNULL constraints on your table query the USER_CONSTRAINTS table.

USER_CONSTRAINTS data dictionary table display following main column which we can frequently use for finding constraint details

Column Name	Description
OWNER	The owner of the constraint
CONSTRAINT_NAME	The name of the constraint
TABLE_NAME	The name of the table associated with the constraint
CONSTRAINT_TYPE	The type of constraint: P:- Primary Key constraint R:- Foreign Key constraint U:- Unique Key constraint C:- Check Constraint
SEARCH_CONDITION	The condition used for CHCK constraints

Note:- Constraints that are not named by the owner receive the system assigned constraint name like (SYS_Cnnnn)

ADDING CONSTRAINT

You can add a constraint for existing tables by using the ALTER TABLE statement with the ADD CONSTRAINT clause.

Syntax :- ALTER TABLE <table name>
ADD CONSTRAINT <constraint name> <constraint definition> (<column name>);

Example :- already given in every constraints.

- GUIDE LINE:-
  - → You can add, drop a constraint, but you can not modify its structure.
  - → You can add a NOT NULL constraint to an existing column by using the MODIFY clause of the ALTER TABLE statement.
  - → you can define a NOT NULL column only if the table is empty or if the column has a value for every row.

DROPING CONSTRAINT

To DROP a constraint, you can identify the constraint name from the USER_CONSTRAINTS and USER_CONS)COLUMNS data dictionary table. Then use the ALTER TABLE statement with DROP clause. By using CASCADE option with the DROP clause any dependent constraints will also be droped.

Syntax:- ALTER TABLE <table name> DROP CONSTRAINT <constraint name>;
Here, constraint name is oracle server given constraint name like (SYS-Cnnnn) OR user defined constraint name

We will focus on the following constraints:

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK

SQL NOT NULL Constraint

The NOT NULL constraint enforces a column to NOT accept NULL values.
The NOT NULL constraint enforces a field to always contain a value. This means that you cannot insert a new record, or update a record without adding a value to this field.
The NOT NULL constraint can define only at column level statement not at table level.

The following SQL enforces the "P_Id" column and the "LastName" column to not accept NULL values:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255)
)
```

SQL UNIQUE Constraint

- The UNIQUE constraint uniquely identifies each record in a database table.
- The UNIQUE and PRIMARY KEY constraints both provide a guarantee for uniqueness for a column or set of columns.
- A PRIMARY KEY constraint automatically has a UNIQUE constraint defined on it.
- Note that you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.
- Unique constraint create automatic index file for identifying unique records.
- Unique constraint allows you null value unless you also define NOT NULL constraints for the same column.
- Unique constraint can be defined at the column or table level. A composite UNIQUE key is created by using the table level definition.

SQL UNIQUE Constraint on CREATE TABLE

The following SQL creates a UNIQUE constraint on the "P_Id" column when the "Persons" table is created:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
UNIQUE (P_Id)
)
```

```
CREATE TABLE Persons
(
P_Id int NOT NULL UNIQUE,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255)
)
```

To allow naming of a UNIQUE constraint, and for defining a UNIQUE constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
CONSTRAINT uc_PersonID UNIQUE (P_Id,LastName)C
)
```

SQL UNIQUE Constraint on ALTER TABLE

To create a UNIQUE constraint on the "P_Id" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons
ADD UNIQUE (P_Id)
```

To allow naming of a UNIQUE constraint, and for defining a UNIQUE constraint on multiple columns, use the following SQL syntax:

```
ALTER TABLE Persons
ADD CONSTRAINT uc_PersonID UNIQUE (P_Id,LastName)
```

To DROP a UNIQUE Constraint

To drop a UNIQUE constraint, use the following SQL:

```
ALTER TABLE Persons
DROP CONSTRAINT uc_PersonID
```

SQL PRIMARY KEY Constraint

- The PRIMARY KEY constraint uniquely identifies each record in a database table.
- Primary keys must contain unique values.
- A primary key column cannot contain NULL values.
- Each table should have a primary key, and each table can have only one primary key.
- Primary key constraint create automatic index file for identifying unique records.
- Primary key constraint can be defined at the column or table level.
- A composite PRIMARY KEY is created by using the table level definition maximum up to 16 columns .

SQL PRIMARY KEY Constraint on CREATE TABLE

The following SQL creates a PRIMARY KEY on the "P_Id" column when the "Persons" table is created:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
Address varchar(255),
City varchar(255),
PRIMARY KEY (P_Id) )
```

```
CREATE TABLE Persons
(
P_Id int PRIMARY KEY,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255)
)
```

To allow naming of a PRIMARY KEY constraint, and for defining a PRIMARY KEY constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
CONSTRAINT pk_PersonID PRIMARY KEY (P_Id,LastName)
)
```

SQL PRIMARY KEY Constraint on ALTER TABLE

To create a PRIMARY KEY constraint on the "P_Id" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons
ADD PRIMARY KEY (P_Id)
```

To allow naming of a PRIMARY KEY constraint, and for defining a PRIMARY KEY constraint on multiple columns, use the following SQL syntax:

```
ALTER TABLE Persons
ADD CONSTRAINT pk_PersonID PRIMARY KEY (P_Id,LastName)
```

Note: If you use the ALTER TABLE statement to add a primary key, the primary key column(s) must already have been declared to not contain NULL values (when the table was first created).

To DROP a PRIMARY KEY Constraint

To drop a PRIMARY KEY constraint, use the following SQL:

```
ALTER TABLE Persons DROP CONSTRAINT pk_PersonID
```

SQL FOREIGN KEY Constraint

A FOREIGN KEY in one table points to a PRIMARY KEY or UNIQUE key in another table.
Let's illustrate the foreign key with an example. Look at the following two tables:
The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	2
4	24562	1

Note that the "P_Id" column in the "Orders" table points to the "P_Id" column in the "Persons" table.
The "P_Id" column in the "Persons" table is the PRIMARY KEY in the "Persons" table.
The "P_Id" column in the "Orders" table is a FOREIGN KEY in the "Orders" table.

- The FOREIGN KEY constraint is used to prevent actions that would destroy link between tables.
- The FOREIGN KEY constraint also prevents that invalid data is inserted into the foreign key column, because it has to be one of the values contained in the table it points to.

- No need to write “Foreign Key” key word at column level statement. The foreign key key word you can write at table level statement only.

SQL FOREIGN KEY Constraint on CREATE TABLE

The following SQL creates a FOREIGN KEY on the "P_Id" column when the "Orders" table is created:

```
CREATE TABLE Orders
(
O_Id int NOT NULL,
OrderNo int NOT NULL,
P_Id int,
PRIMARY KEY (O_Id),
FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
)
```

```
CREATE TABLE Orders
(
O_Id int NOT NULL PRIMARY KEY,
OrderNo int NOT NULL,
P_Id int REFERENCES Persons(P_Id)
)
```

To allow naming of a FOREIGN KEY constraint, and for defining a FOREIGN KEY constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Orders
(
O_Id int NOT NULL,
OrderNo int NOT NULL,
P_Id int,
PRIMARY KEY (O_Id),
CONSTRAINT fk_PerOrders FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
)
```

SQL FOREIGN KEY Constraint on ALTER TABLE

To create a FOREIGN KEY constraint on the "P_Id" column when the "Orders" table is already created, use the following SQL:

```
ALTER TABLE Orders
ADD FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
```

To allow naming of a FOREIGN KEY constraint, and for defining a FOREIGN KEY constraint on multiple columns, use the following SQL syntax:

```
ALTER TABLE Orders
ADD CONSTRAINT fk_PerOrders FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
```

To DROP a FOREIGN KEY Constraint

To drop a FOREIGN KEY constraint, use the following SQL:

```
ALTER TABLE Orders
DROP CONSTRAINT fk_PerOrders
```

SQL CHECK Constraint

- The CHECK constraint is used to limit the value range that can be placed in a column.
- If you define a CHECK constraint on a single column it allows only certain values for this column.
- If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.
- CHECK constraints uses all conditional operator which you can use with the where clause.

SQL CHECK Constraint on CREATE TABLE

The following SQL creates a CHECK constraint on the "P_Id" column when the "Persons" table is created. The CHECK constraint specifies that the column "P_Id" must only include integers greater than 0.

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
CHECK (P_Id>0) )
```

```
CREATE TABLE Persons
(
P_Id int NOT NULL CHECK (P_Id>0),
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255)
)
```

To allow naming of a CHECK constraint, and for defining a CHECK constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
CONSTRAINT chk_Person CHECK (P_Id>0 AND City='Sandnes')
)
```

SQL CHECK Constraint on ALTER TABLE

To create a CHECK constraint on the "P_Id" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons
ADD CHECK (P_Id>0)
```

To allow naming of a CHECK constraint, and for defining a CHECK constraint on multiple columns, use the following SQL syntax:

```
ALTER TABLE Persons
ADD CONSTRAINT chk_Person CHECK (P_Id>0 AND City='Sandnes')
```

To DROP a CHECK Constraint

To drop a CHECK constraint, use the following SQL:

```
ALTER TABLE Persons
DROP CONSTRAINT chk_Person
```

SQL DEFAULT

The DEFAULT is used to insert a default value into a column. The default value will be added to all new records, if no other value is specified.

SQL DEFAULT on CREATE TABLE

The following SQL creates a DEFAULT constraint on the "City" column when the "Persons" table is created:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255) DEFAULT 'Sandnes' NOT NULL)
```

The DEFAULT constraint can also be used to insert system values, by using functions like SYSDATE() without other constraint:

```
CREATE TABLE Orders
(
O_Id int NOT NULL,
OrderNo int NOT NULL,
P_Id int,
OrderDate date DEFAULT SYSDATE()
)
```

SQL DEFAULT Constraint on ALTER TABLE

To create a DEFAULT constraint on the "City" column when the table is already created, use the following SQL:

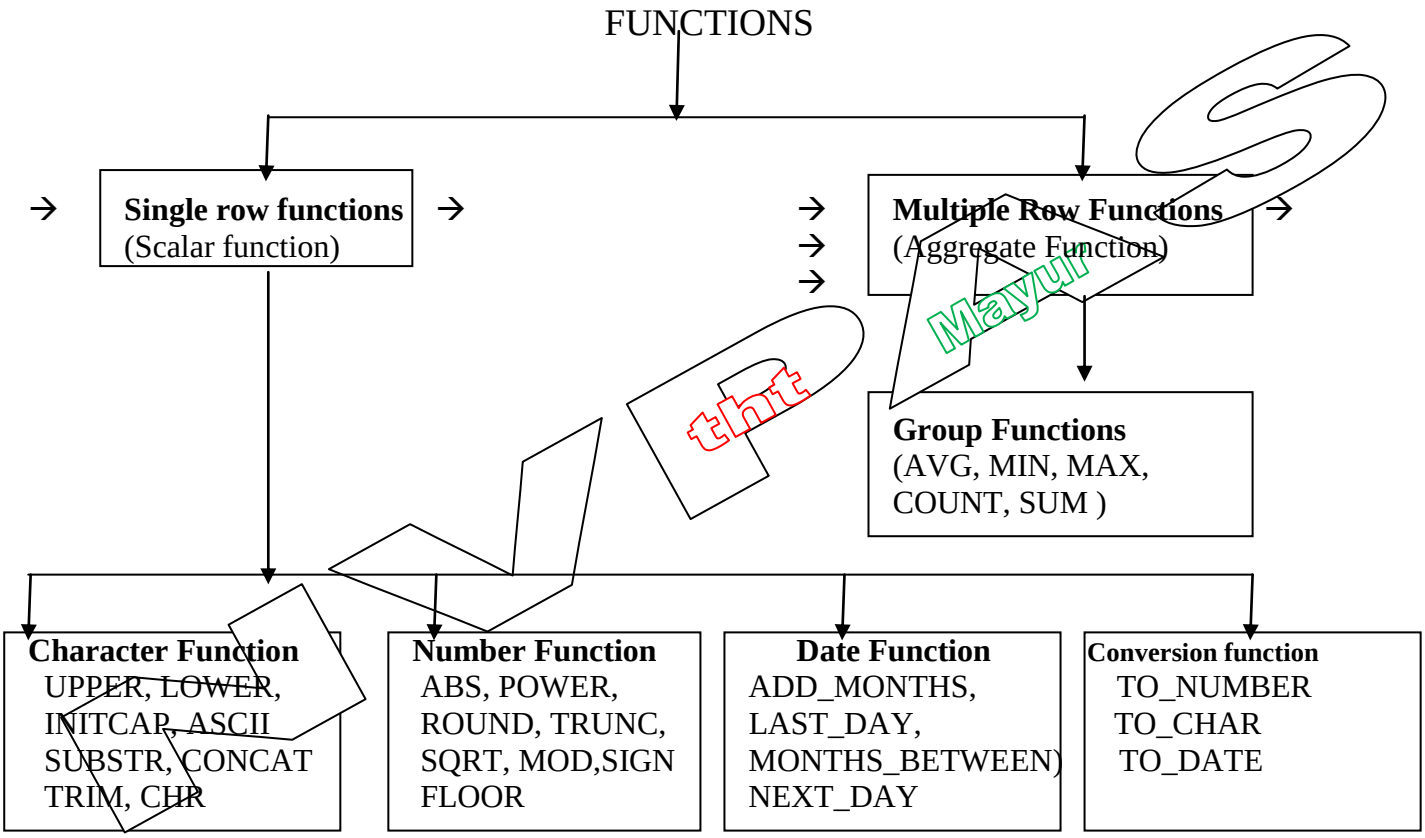
```
ALTER TABLE Persons
ALTER COLUMN City SET DEFAULT 'SANDNES'
```

To DROP a DEFAULT Constraint

```
ALTER TABLE Persons
ALTER COLUMN City DROP DEFAULT
```

FUNCTIONS: Functions are similar to operators in that they manipulate data items and return a result. Functions differ from operators in the format of their arguments. This format enables them to operate on zero, one, two, or more arguments:
function(argument, argument, ...)

A function without any arguments is similar to a pseudocolumn. However, a pseudocolumn typically returns a different value for each row in the result set, whereas a function without any arguments typically returns the same value for each row.



SINGLE ROW FUNCTIONS (SCALAR FUNCTION) :

These functions operate on single rows only and return one result per row. These are used to manipulate data items . They accept one or more arguments and return one value for each row returned by the query an argument can be one of the following.

→ User supplied constant.
→ Column name
→ Expression

- FEATURES :

→ Acting on each row returned in the query
→ Returning one result per row
→ Possibly returning a data value of a different type than that referenced.
→ Possibly expecting one or more arguments
→ Can be used in SELECT , WHERE and ORDER BY Clause

CHARACTER FUNCTIONS:

IT accepts character input and can return both character and number value. This is also divide into two part like 1) Case manipulation 2) Character manipulation

1) Case manipulation : Though these functions you can change the cases of the character data type value. The functions are

→ UPPER :- To convert character into upper case.
→ LOWER:- To convert character into lower case.
→ INITCAP :- To convert character into initial capital and rest in lower case.

Syntax: UPPER(Col/Expr), LOWER (Col/Expr), INITCAP (Col/Expr)
Example :- Select upper(ename), lower(' HOW are You'), INITCAP(ename) from emp;

2) Character manipulation : Though these functions you can manipulate the character and get required output.

→ **SUBSTR :-** It extract a string of determined length. Returns a portion of character, beginning at character m, and going up to character n. If n is omitted result returns up to the last character.

Syntax :- (<col / expr / user supplied string>, <start position>, <length n>)
Where , **String** is the source string, S**tart position** is the position for extraction., **Length** is the no of character to extract.)
Example :- Select substr('HOW are You', 5,3) "substring" from dual;
Output :- 'are'

→ **LTRIM :-** It removes characters from the left of char with initial character removed upto the first character not in set.
Syntax :- LTRIM (col / expr, char set)
Example :- select LTRIM(ename, ' ') from emp;
This statement removes blank space from the left side

→ **RTRIM :-** It removes char, with final characters removed after the last character not in the set. Char set is optional default is spaces.
Syntax :- RTRIM (col / expr, char set)
Example :- Select RTRIM('ISHAN', 'N')
Result :- ISHA

→ **CHR :-** SQL CHR function is the opposite of ASCII. It converts an integer in range 0-255 into a ascii character.
Syntax:- CHR(Expression)
Example :- Select CHR(65), CHR(97) from dual;
Result :- A , a

→ **ASCII :-** If you want to find a numerical value of a character which is in range of 0 to 255 you can use SQL ASCII function. If you pass a NULL string to ASCII function it will return NULL. If the string is empty, the function will return 0. Any character which its range is out of 0 to 255, the ASCII function will return an error
Syntax :- ASCII(Expression)
Example :- Select ASCII('a'), ASCII('A') from dual
Result :- 97 , 65

→ **CONCAT :-** SQL CONCAT() function is used to concatenate two or more strings.
Syntax :- CONCAT(Col/Expression, Col/ Expression)
Example :- Select CONCAT(Ename, ' : ' , Sal) from emp
Result :- jack : 15000
Scott : 20000

NUMBER FUNCTION

It accepts numeric as input and return numeric value

→ **ABS :-** It returns the absolute value of 'n'
Syntax :- ABS(n)
Example :- select ABS(-15) from dual
Result :- 15

→ **POWER :-** It returns m raised to the n^{th} power. N must be an integer value, else an error is returned.
Syntax :- POWER (m,n)
Example :- select power(3,3) "Raised" from dual
Result :- 27

→ **ROUND :-** The ROUND functions rounds the column, expressions, or value to 'n' decimal places. If the second argument 0 or is missing, the value is rounded to zero decimal places. If the second argument 2 the value is rounded to two decimal places. If the second argument is -2, the value is rounded to two decimal places to the left. This function can also be used with date data type.
Syntax :- ROUND (col / expr, n^{th} decimal places)
Example :- Select ROUND (45.923,2), ROUND (45.923,0), ROUND (45.923,-1) from dual
Result:- 45.92 , 46, 50

→ **TRUNC :-** It returns a number truncated to a certain number of decimal places. The decimal place value must be an integer. If this parameter is omitted, the TRUNC function will truncate the number to 0 decimal places.
Syntax :- TRUNC (col / expr, n^{th} decimal places)
Example :- Select TRUNC(45.923,2), TRUNC(45.923,0), TRUNC(45.923,-2) from dual
Result :- 45.92 , 45, 0

→ **SQRT :-** It returns the square root of 'n' number. If n<0, NULL value will be return.
Syntax :- Select SQRT(625) from dual;
Result :- 25

→**FLOOR :-** It returns largest integer equal to or less than *n*. This function takes as an argument any numeric datatype or any nonnumeric datatype that can be implicitly converted to a numeric datatype. The function returns the same datatype as the numeric datatype of the argument.
Syntax :- FLOOR (n)
Example :- Select floor(15.7) From dual;
Ans:- 15

→**MOD :-** It returns the remainder of n2 divided by n1. Returns n2 if n1 is 0.
Syntax :- MOD (n2, n1)
Example :- Select MOD(11, 4) From dual;
Ans:- 3

→**SIGN :-** It returns the sign of n. For value of NUMBER type, the sign is:

- -1 if $n<0$, 0 if $n=0$, 1 if $n>0$

Syntax :- SIGN (n)
Example :- Select SIGN(-15) From dual;
Ans:- -1

DATE FUNCTION

It accepts date as input and return date or numeric value

Oracle database stores dates in an internal numeric format, representing the Century, year, month, day, hours, minutes and seconds. When a record with a date column is inserted into a table the century information is picked up from the SYSDATE function. The DATE datatype always stores year information as a four digit number internally.

→ SYSDATE :- SYSDATE is a date function that returns the current database server date and time.

Since the database stores dates as number, you can perform calculation using arithmetic operators such as addition and subtraction. You can add and subtract number constant as well as date.

OPERATION	RESULT	DESCRIPTION
Date + number	Date	Adds a number of days to a date
Date – number	Date	Subtract a number of days from date.
Date – Date	Number of days	Subtract one date from other date
Date + number/24	Date	Adds a number of hours to date

Example :- Select ename, (Sysdate – hire_date)/7 as week from emp;
Result :- Display no. of weeks employees has worked.

→ If a higher date is subtracted from an older date, the difference is a negative number.

→**ADD_MONTHS :-** Adds 'n' number of calendar months to date. The value of 'n' must be an integer and can be negative .
Syntax :- ADD_MONTHS (date, n)
Example :- Select ADD_MONTHS('11-Jan-94', 6) from dual;
Result:- '11- Jul-94'

→ **LAST_DAY :-** Finds the date of the last day of the month that contains date.
Syntax :- LAST_DAY(date)
Example :- Select LAST_DAY('01-FEB-95')
Result :- 28-FEB-95

→ **MONTHS_BETWEEN :-** It finds the number of months between date1 and date2. The result can be positive or negative.
Syntax :- MONTHS_BETWEEN(date1, date2)
Example :- Select ename, MONTHS_BETWEEN(sysdate, hire_date) from emp;
Result :- It display the employees name with the number of month's they have worked.

→ **NEXT_DAY :-** NEXT_DAY returns the date of the first weekday named by *char* that is later than the date *date*. The return type is always DATE, regardless of the datatype of *date*. The argument *char* must be a day of the week in the date language of your session, either the full name or the abbreviation.

Syntax :- NEXT_DAY(date, char)
Example :- SELECT NEXT_DAY('02-FEB-2001','TUESDAY') "NEXT DAY" FROM DUAL;
Result :- 06-Feb-2001

CONVERSION FUNCTION

This is also again divide into two part 1) Implicit data type and 2) Explicit data type conversion.

In some cases, Oracle server uses data of one data type where it expects data of a different data types when this happens, Oracle server can automatically convert the data to the expected data type which is known as implicit data type conversion.

Explicit data type conversions are done by using the conversion functions. Conversion function convert a value from one data type to another.

Date Format:

ELEMENTS	DESCRIPTION
YYY or YY or Y	Last three, two one digit of the year.
MM	Month's two digit value
MONTH	Name of the month padded with blanks to length of nine character
MON	Name of month, three letter abbreviation.
DDD or DD or D	Day of year, month or a week
DAY	Name of day padded with blanks to a length of nine characters.
DY	Name of day; three letter abbreviation.
AM or PM	Meridian indicator
A.M. or P.M.	Meridian indicator with (.) dot.
HH or HH12 or HH24	Hours of day, or hour(1-12), or hour(0-23)
MI	Minutes (0-59)
SS	Seconds (0-59)
" of the "	Quoted string is reproduced in the result
DDTH	Ordinal number (4^{TH})
DDSP	Spelled out number (FOUR)
DDSPTH or THSP	Spelled out ordinal numbers (FOURTH)

Number Format:

ELEMENTS	DESCRIPTION	EXAMPLE	RESULT
9	Numeric position(Number of 9s determine display)	999999	123456
0	Display leading zeros	099999	001234
$	Floating dollar sign	$99999	$12345
.	Decimal point in position specified	9999.99	1234.00
,	Comma in position specified	999,999	1,234

→ **TO_CHAR :-** This function facilitate the retrieval of data in a format different from default format. It can be used to convert number as well as date data type into character or varchar2 data type. When working with number values such as character strings. You should convert those numbers to the character data type using TO_CHAR function, which translates a value of NUMBER data type to varchar2 data type. This technique is specially useful with concatenation.

Syntax :- TO_CHAR (<date or number col / expr>, 'format model')
Example :- Select TO_CHAR(salary, ' S99,999.00') salary,
TO_CHAR(hire_date, 'DDSP-MONTH-YY') from emp;
Result :- It display salary and hiredate of the employees in specified format.

→ T**O_DATE :-** It converts a char value into a date value. It allows a user to insert date into a date column in any required format, by specifying the character value of the date to be inserted and its format.
Syntax:- TO_DATE(Char col / expr, 'format model')
Example :- Select TO_DATE('28-DECEMBER-01','DD-MONTH-YY') from dual
Result :- convert your character text into actual date.

→ **TO_NUMBER :-** converts char, a CHARACTER value expressing a number to number data type
Syntax :- TO_NUMBER(char col / expr)
Example :- Select TO_NUMBER(SUBSTR('$100', 2,3)) from dual;
Result :- 100 (actual number)

MULTIPLE ROW FUNCTIONS OR AGGREGATE FUNCTIONS

Functions can manipulate group of rows to give one result per group of rows. These are known as group functions.

GROUP FUNCTIONS

Unlike Single row functions operate on set of rows to give one result per group. These sets may be the whole table or the table split into group.

→ You can use the MIN , MAX function for any data type column or expression.
→ AVG, SUM functions can only use with numeric data types.
→ All group functions ignores null values.

→ **AVG :-** It returns the average value of column or expression or 'n' numbers. It ignores null values.
Syntax :- AVG(col/expr/n)
Example :- Select AVG(Sal) from emp;
Result :- It display the average salary of the all employees.

→ **MIN :-** It returns the minimum value of expression. It ignores null values.
Syntax :- MIN(col/expr)
Example :- Select MIN(Sal) from emp;
Result :- It display the minimum salary from the all employees salaries.

→ **MAX :-** It returns the maximum value of expression. It ignores null values.
Syntax :- MAX(col/expr)
Example :- Select MAX(Sal) from emp;
Result :- It display the maximum salary from the all employees salaries.

→ **SUM :-** It returns total values of column or expression or 'n' numbers. It ignores null values.

Syntax :- SUM (col / expr / n)

Example :- Select SUM(Sal) from emp;

Result :- It display the Total salary of the all employees salary.

→ **COUNT :-**

COUNT(*) returns the number of rows in a table that satisfy the criteria of the SELECT statement including duplicate rows and rows containing null values in any of the column.

COUNT(col/expr) returns the number of non null values in the column identified by expr.

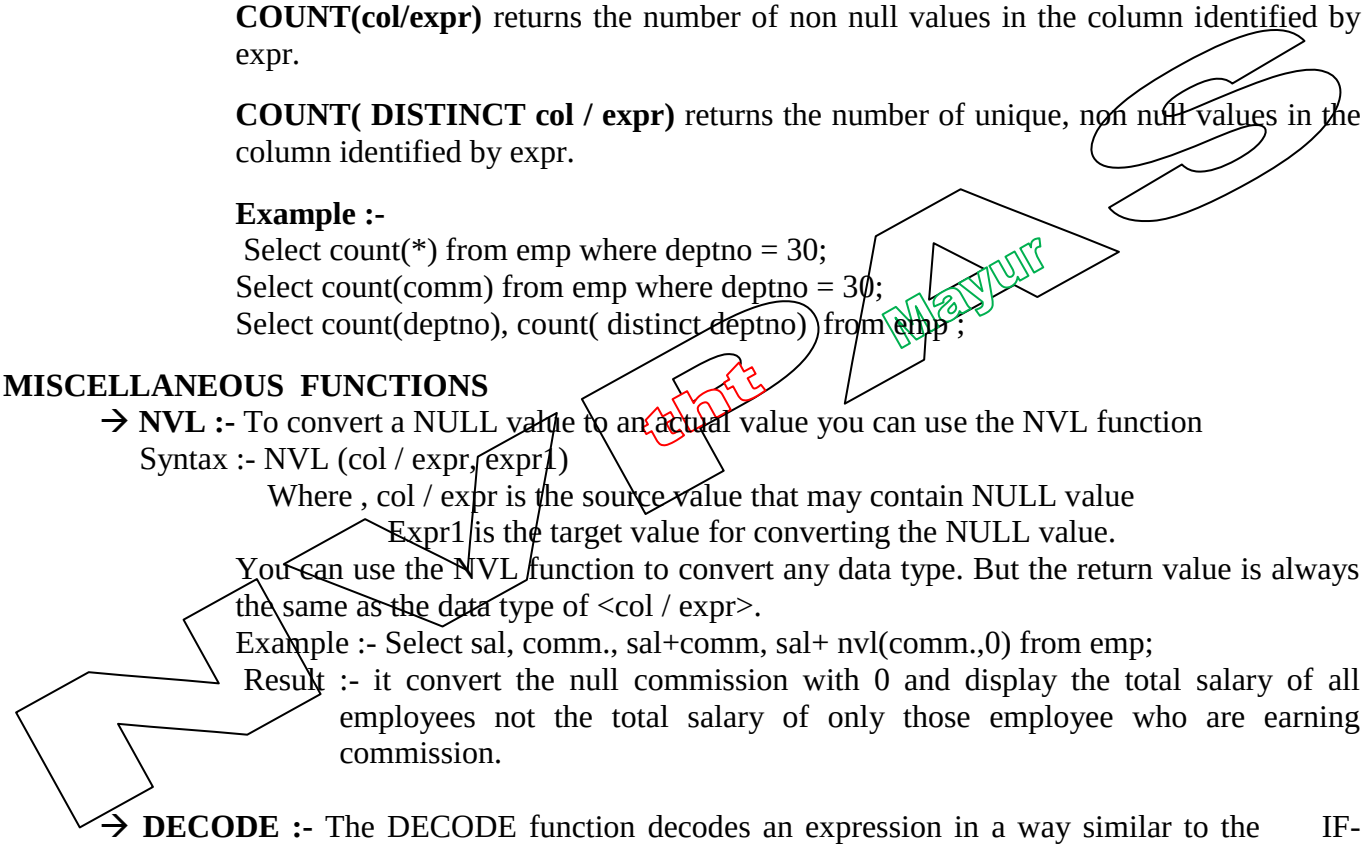
COUNT(DISTINCT col / expr) returns the number of unique, non null values in the column identified by expr.

Example :-

Select count(*) from emp where deptno = 30;

Select count(comm) from emp where deptno = 30;

Select count(deptno), count(distinct deptno) from emp ;



MISCELLANEOUS FUNCTIONS

→ **NVL :-** To convert a NULL value to an actual value you can use the NVL function

Syntax :- NVL (col / expr, expr1)

Where , col / expr is the source value that may contain NULL value

Expr1 is the target value for converting the NULL value.

You can use the NVL function to convert any data type. But the return value is always the same as the data type of <col / expr>.

Example :- Select sal, comm., sal+comm, sal+ nvl(comm.,0) from emp;

Result :- it convert the null commission with 0 and display the total salary of all employees not the total salary of only those employee who are earning commission.

→ **DECODE :-** The DECODE function decodes an expression in a way similar to the IF-THEN-ELSE logic used in various languages. The DECODE function decodes expression after comparing it to each search value. If the expression is the same as search, result is performed or display. If the default is omitted, a null value is returned where a search value does not match with any value of the result values.

Syntax :- DECODE (col / expr, search1, result1, search2, result2, default)

Example :- Select name, design, sal, decode(design, 'IT_PROG',1.10*sal, 'ST_CLERK', 1.15*sal, 'SA_REP', 1.20*sal, Sal) revised_salary from emp;

Result:- Above decode function revised the salary on the conditional base like

If designation is IT_PROG then increase salary 10%

If designation is ST_CLERK then increase salary 15%

If designation is SA_REP then increase salary 20%

→ **COALESCE**: COALESCE returns the first non-null *expr* in the expression list. At least one *expr* must not be the literal NULL. If all occurrences of *expr* evaluate to null, then the function returns null.

Syntax : Coalesce (expr1, expr2, ...n)

Example : SELECT product_id, list_price, min_price,
COALESCE(0.9*list_price, min_price, 5) "Sale"
FROM product_information WHERE supplier_id = 102050;

Result : If the list_price column has vaue than it will display list_price*0.9.
If the list_price column contains null value then result will be the min_price.
If the list_price and min_price both has null value then result will be the 5.